

FH JOANNEUM - University of Applied Sciences

Dynamic Resource Allocation and Scaling of Stateful Workloads in Kubernetes

Bachelorarbeit

zur Erlangung des akademischen Grades

„Bachelor of Science in Engineering“

eingereicht am Studiengang Wirtschaftsinformatik

Studienrichtung: Wirtschaftsinformatik

Verfasst von: Johann Krischan Hipolito

Betreuung: Michael Nestler

Graz, 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	i
Abbildungsverzeichnis	ii
Tabellenverzeichnis	iii
Abkürzungsverzeichnis	iv
Symbolverzeichnis	v
Formelverzeichnis	vi
Kurzfassung	vii
Abstract	vii
1 Introduction	1
1.1 Cloud Computing and the Rise of Container Orchestration	1
1.2 Kubernetes as the Standard for Container Orchestration	1
1.3 The Challenge of Scaling Stateful Applications	2
1.4 Autoscaling Strategies: From Reactive to Predictive	2
1.5 Research Objective and Thesis Structure	3
2 Scaling Stateful and Stateless Applications in Kubernetes	4
2.1 Stateless Applications and Horizontal Scalability	4
2.2 The Challenges of Scaling Stateful Applications	4
3 Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads	7
4 Technologies	9
4.1 K3s	9
4.2 FastAPI	9
4.3 Redis Cluster	9
4.4 Horizontal Pod Autoscaler (HPA)	10
4.5 KEDA	10
4.6 Prometheus and Grafana	10
4.7 Helm	11
4.8 Grafana k6	11
5 Experiment Architecture	12
6 Results and Analysis	13
7 Conclusion	14
Anhang	15
Glossar	16
Literaturverzeichnis	17
Stichwortverzeichnis	19
Anlagen	20

Abbildungsverzeichnis

Tabellenverzeichnis

Abkürzungsverzeichnis

Hier steht das Abkürzungsverzeichnis.

Symbolverzeichnis

Hier steht das Symbolverzeichnis.

Formelverzeichnis

Hier steht das Formelverzeichnis.

Kurzfassung

Hier steht die Kurzfassung auf Deutsch.

Abstract

Here is the abstract in English.

1 Introduction

1.1 Cloud Computing and the Rise of Container Orchestration

The rapid evolution of modern software systems has fundamentally transformed the way infrastructure is designed, deployed, and operated. At the heart of this transformation lies *cloud computing* — a paradigm that enables on-demand access to a shared pool of configurable computing resources, including networks, servers, storage, applications, and services, that can be rapidly provisioned and released with minimal management effort (Mell und Grance, 2011). This model has shifted the industry away from static, on-premises data centres toward elastic, scalable environments that can react dynamically to workload demands.

As organisations increasingly adopt cloud-native architectures, the notion of designing systems that are resilient, scalable, and operationally excellent has become central to engineering practice. The AWS Well-Architected Framework, for instance, codifies these concerns into a set of established design principles for container-based workloads, emphasising repeatability, automation, and observability as first-class concerns in modern infrastructure (Amazon Web Services, 2022).

A critical enabler of cloud-native architectures is the *container* — a lightweight, self-contained unit of software that packages application code together with its dependencies, configuration, and runtime environment. Unlike traditional virtual machines, containers share the host operating system kernel and therefore incur significantly less overhead while still providing process-level isolation (Docker Inc., 2016). This characteristic makes containers especially well suited to microservice architectures, where large monolithic applications are decomposed into smaller, independently deployable services that can be scaled individually based on demand.

1.2 Kubernetes as the Standard for Container Orchestration

Managing containerised workloads at scale introduces significant operational complexity. Scheduling containers across a fleet of machines, maintaining desired application state, performing rolling updates, and recovering from failures all require robust automation. It is in response to exactly these challenges that *Kubernetes* — also referred to as K8s — emerged as the dominant container orchestration platform.

Originally designed and used internally at Google as the *Borg* system, Kubernetes was open-sourced in 2014 and donated to the Cloud Native Computing Foundation (CNCF), where it has since grown into the second-largest open-source project in the world (IBM, 2024). Kubernetes provides a declarative API for defining the desired state of an application cluster; its control plane continuously reconciles the observed state of the system against this desired state, automatically scheduling pods, managing service discovery, and orchestrating storage (Kubernetes Documentation, 2024c).

To reduce operational complexity further, lightweight Kubernetes distributions such as K3s — developed by Rancher and certified by the CNCF — have gained widespread

adoption. K3s packages the full Kubernetes feature set into a single binary with a significantly reduced memory footprint, making it suitable for edge computing, IoT environments, and resource-constrained infrastructure (K3s Project, 2024). In the context of this thesis, K3s serves as the underlying cluster management layer for the experimental infrastructure under investigation.

1.3 The Challenge of Scaling Stateful Applications

While Kubernetes excels at managing stateless workloads — where any replica of a service can handle any request independently — scaling *stateful applications* presents a distinct set of challenges. Stateful applications, such as in-memory databases and message queues, maintain persistent data or internal state that must be carefully managed across the lifecycle of individual pods. Kubernetes provides the `StatefulSet` primitive to address these requirements, offering stable network identities, ordered deployment and scaling, and persistent volume binding (Kubernetes Documentation, 2024e).

Redis is a canonical example of such a stateful application. It is a high-performance, in-memory data store widely used for caching, session management, and real-time analytics (Redis Ltd., 2024a). When deployed as a Redis Cluster across multiple nodes, it uses sharding to distribute data and replication to ensure fault tolerance. Scaling a Redis Cluster in a Kubernetes environment is non-trivial: adding or removing nodes requires coordinated resharding of data slots, careful management of cluster topology, and maintenance of quorum — operations that introduce significant risk if performed reactively under resource pressure.

Recent work from the CNCF community has highlighted these concerns, noting that while Kubernetes operators can partially automate lifecycle management for Redis, the path toward robust, cloud-native stateful services remains an open challenge (Cloud Native Computing Foundation, 2024).

1.4 Autoscaling Strategies: From Reactive to Predictive

Kubernetes provides native support for horizontal scaling through the *Horizontal Pod Autoscaler* (HPA), which monitors CPU and memory utilisation metrics and adjusts the number of pod replicas accordingly (Kubernetes Documentation, 2024a). While the HPA is effective for stateless services under gradually increasing load, its reactive nature introduces an inherent latency: scaling decisions are made only *after* resource thresholds have already been breached. For latency-sensitive or stateful workloads, this lag can result in degraded performance, pod restarts, or cascading failures during traffic spikes (Thoras, 2024).

An increasingly popular alternative is *Kubernetes Event-Driven Autoscaling* (KEDA), a CNCF-graduated project that extends the standard HPA to support event- and metric-driven scaling from a rich ecosystem of external sources, including message queues, databases, and custom Prometheus metrics (KEDA Project, 2024a; Cloud Native Computing Foundation, 2023). By decoupling scaling decisions from simple resource utili-

sation and enabling them to respond to application-level events, KEDA enables a more proactive and context-aware approach to scaling. This is particularly relevant for stateful workloads where pre-emptive resource provisioning may prevent the cluster-level disruptions that reactive scaling cannot avoid.

1.5 Research Objective and Thesis Structure

This thesis investigates the performance differences between two autoscaling strategies for stateful applications deployed in a Kubernetes cluster managed by K3s. The first strategy employs the native Horizontal Pod Autoscaler in a *reactive* configuration, scaling a Redis Cluster in response to CPU and memory resource expenditure. The second strategy employs a *custom KEDA-based operator* in a predictive, event-driven configuration, triggering scaling decisions based on application-level metrics sourced from a Prometheus observability stack.

The experimental infrastructure consists of a FastAPI-based endpoint service (FastAPI, 2024b), a Redis Cluster with a minimum of three nodes, an observability stack comprising Prometheus and Grafana deployed via Helm, and two scaling configuration manifests representing the naive HPA and KEDA-driven approaches respectively. All components are deployed on a K3s cluster, providing a reproducible and resource-constrained environment representative of real-world edge and on-premises deployments.

The remainder of this thesis is structured as follows. Chapter 2 details the differences between the automatic provisioning of stateless and stateful applications, as well as the issues that arise when scaling stateful workloads. Chapter 4 provides the theoretical background on Kubernetes architecture, autoscaling mechanisms, stateful workloads, and the Redis Cluster model. Chapter 5 describes the system design and experimental setup. Chapter 6 presents the experimental results and performance analysis. Chapter 7 summarises the findings and discusses directions for future work.

2 Scaling Stateful and Stateless Applications in Kubernetes

A fundamental distinction in distributed systems design is that between *stateless* and *stateful* applications, and this distinction has direct consequences for how workloads are deployed, managed, and scaled inside a Kubernetes cluster. Understanding this divide is essential for motivating the core research question of this thesis.

2.1 Stateless Applications and Horizontal Scalability

A stateless application does not retain any client-specific data between requests. Each request is self-contained and can be served by any replica of the application without prior knowledge of previous interactions (Red Hat, 2025). This architectural property makes stateless services trivially scalable: because every replica is functionally identical, the Kubernetes scheduler can start or terminate any number of them at any time without coordination. Kubernetes Deployments and ReplicaSets are the canonical workload primitives for stateless applications, providing identical, interchangeable pod replicas with no notion of stable identity or ordered lifecycle (Kubernetes Documentation, 2024e).

Horizontal scaling of a stateless service via the Horizontal Pod Autoscaler (HPA) is therefore straightforward: when aggregate CPU or memory utilisation exceeds a configured threshold, the HPA controller issues a scale-out instruction to the relevant Deployment, and the new replicas are scheduled and become available without any data migration, topology reconfiguration, or disruption to existing replicas (Kubernetes Documentation, 2024a). Conversely, when load decreases, replicas can be terminated immediately since no in-replica state is lost. This symmetry between scale-out and scale-in is what makes the reactive HPA model well-suited to stateless workloads.

2.2 The Challenges of Scaling Stateful Applications

Stateful applications, by contrast, maintain persistent data or internal state that is specific to individual instances and must survive pod restarts, rescheduling events, and scaling operations. This category includes relational databases, key-value stores, message queues, and consensus systems. Kubernetes provides the StatefulSet workload primitive to accommodate these requirements. Unlike a Deployment, a StatefulSet guarantees that each pod receives a stable, unique network identity (e.g., `redis-0`, `redis-1`), a dedicated PersistentVolumeClaim that is bound across restarts, and an ordered, sequential lifecycle for pod creation and termination (Kubernetes Documentation, 2024e, Portworx, 2024).

These guarantees, while necessary for correctness, fundamentally complicate autoscaling. The following challenges are particularly relevant in the context of this thesis:

Ordered deployment and termination. `StatefulSet` pods are created and deleted in strict ordinal order. When scaling out from n to $n + 1$ replicas, Kubernetes will not proceed until the n -th pod is fully *Ready*. This serialisation ensures application-level invariants (such as cluster quorum) are maintained but introduces latency that can be problematic under sudden load spikes (Kubernetes Documentation, 2024e). A reactive autoscaler such as the HPA, which fires only after a resource threshold has already been breached, is therefore poorly positioned to absorb burst traffic for stateful workloads: by the time new replicas are ordered, scheduled, initialised, and joined to the cluster, the performance degradation may already be severe.

Persistent volume provisioning. Each new pod added to a `StatefulSet` requires a new `PersistentVolumeClaim` to be dynamically provisioned by the storage backend. Depending on the underlying storage class and infrastructure, this provisioning step can itself introduce non-trivial latency, further extending the time-to-ready for new stateful replicas compared to stateless counterparts (Portworx, 2024).

Cluster topology and data rebalancing. For distributed databases and in-memory stores that shard data across their nodes — such as Redis Cluster — adding or removing a node is not merely a matter of starting or stopping a process. Redis Cluster partitions its keyspace across exactly 16 384 hash slots, each of which is assigned to a primary node (Redis Ltd., 2024b). When a new node is added, the cluster operator must *reshard* a subset of hash slots from existing nodes to the new one; when a node is removed, its slots must first be migrated away. This resharding process involves live data movement between nodes and, if performed reactively under resource pressure, carries a non-trivial risk of increased latency, partial unavailability, or data inconsistency (Google Cloud, 2023).

Quorum and availability constraints. Many stateful distributed systems depend on a quorum of nodes being available to elect a leader, acknowledge writes, or service reads. Redis Cluster requires at least one primary per shard to be available, and Kubernetes `PodDisruptionBudget` (PDB) resources are the mechanism used to express minimum availability constraints during voluntary disruptions such as autoscaling events (Kubernetes Documentation, 2024b). Aggressive scale-in triggered by a reactive autoscaler can violate quorum constraints if PDBs are not carefully configured, risking full cluster unavailability.

The HPA and stateful workloads. While the HPA technically supports `StatefulSets` as scaling targets, its reactive, threshold-based model is widely considered inappropriate for database and cache workloads (Vazquez, 2026). The HPA operates on *lagging indicators*: it reacts to resource pressure that has already materialised. For a stateful application such as Redis, where high memory pressure can degrade cache hit rates and high CPU pressure can stall replication, by the time the HPA observes a breach and a new node has completed resharding, a significant window of degraded performance will have already elapsed. Furthermore, the HPA's default synchronisation period of

15 seconds (KEDA Project, 2024d) means that multiple polling cycles may pass before a scaling decision is acted upon, compounding the lag.

These challenges collectively motivate the investigation of a more *proactive*, event-driven autoscaling strategy for stateful workloads, as explored in this thesis through the use of KEDA.

3 Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads

The limitations of the reactive HPA model described in Section 2.2 motivate the central hypothesis of this thesis: that a *proactive, event-driven autoscaling strategy*, implemented through a custom Kubernetes operator in combination with KEDA, is a superior method of scaling stateful workloads compared to the native HPA baseline.

The Operator Pattern as a Foundation for Stateful Automation. The Kubernetes *operator pattern* was conceived precisely to address the gap between the generic automation that Kubernetes provides out of the box and the deep, application-specific knowledge required to manage complex stateful systems reliably. According to the official Kubernetes documentation, the operator pattern „aims to capture the key aim of a human operator who is managing a service or set of services“, encoding that operational expertise into a software controller that participates in the standard Kubernetes reconciliation loop (Kubernetes Documentation, 2024d). Operators extend the Kubernetes API through *Custom Resource Definitions* (CRDs) and an accompanying controller that continuously reconciles the observed state of the custom resource against its declared desired state.

Critically, the CNCF Operator White Paper explicitly acknowledges that „Kubernetes primitives were not built to manage state by default“ and that „relying on Kubernetes primitives alone brings difficulty managing stateful application“ lifecycle (CNCF TAG App Delivery, 2023). The operator pattern is therefore not merely a convenience — it is the architecturally recommended approach for encoding domain-specific concerns such as data resharding, replication topology maintenance, and quorum-safe scaling into a Kubernetes-native automation layer. For a Redis Cluster deployment, a custom operator is capable of expressing and enforcing exactly the application-level invariants that a generic HPA controller cannot: for example, ensuring that a scale-out event is only issued once the cluster is fully healthy, and that the requisite resharding of hash slots is coordinated atomically with the scheduling of new `StatefulSet` pods.

KEDA’s Event-Driven Model as a Proactive Trigger. Complementing the operator pattern, KEDA introduces an event-driven trigger layer that decouples scaling decisions from raw infrastructure metrics. By consuming application-level signals exposed via Prometheus — such as request rate, cache hit ratio, or queue depth — KEDA’s `ScaledObject` can issue a scaling recommendation *before* CPU or memory thresholds are breached, effectively shifting the scaling decision upstream in the load curve (KEDA Project, 2024d). This is in direct contrast to the HPA model, in which scaling decisions are always downstream of resource exhaustion. The additional configurability of KEDA’s `ScaledObject` — including fine-grained control over polling intervals, cooldown periods, and minimum replica counts (Medium / emreblblvv, 2024) — further allows the autoscaling behaviour to be tuned to the specific performance characteristics of a Redis Cluster, where aggressive scale-in during low-traffic periods is inadvisable due to the cost of subsequent resharding.

3. Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads

Supporting Evidence from the Literature. The hypothesis is supported by a growing body of academic and industry research. Mondal et al. identified that the HPA's reliance on CPU and memory as lagging indicators makes it „incapable of foreseeing upcoming workload spikes“, leading to Quality of Service violations, long-tail latency, and wasted resources (Wanigasooriya und Ekanayake, 2026). Wanigasooriya and Ekanayake similarly demonstrated in the NimbusGuard study that proactive autoscaling frameworks „translate into superior performance and cost efficiency compared to existing reactive methods“ when evaluated alongside standard HPA and KEDA baselines under identical, phased load patterns (Wanigasooriya und Ekanayake, 2026). Although that study positions KEDA itself as part of the reactive baseline relative to a deep-learning agent, it nevertheless confirms that event-driven, application-aware autoscaling consistently outperforms pure resource-metric scaling — a finding that directly supports the use of KEDA-sourced Prometheus metrics as the trigger mechanism in the present thesis.

Furthermore, Toka et al. identified three structural drawbacks in the current Kubernetes reactive scaling process: its purely observation-based nature, its inability to adapt to demand variability, and the burden of manually tuning numerous threshold parameters (Wanigasooriya und Ekanayake, 2026). A KEDA-based implementation driven by application-level Prometheus metrics partially addresses all three concerns: it provides an application-aware trigger rather than an infrastructure-level observation, it can be parameterised against workload-specific signals that vary with the demand pattern of the Redis Cluster, and it reduces manual threshold tuning by anchoring scaling decisions to semantically meaningful metrics.

Scope and Expected Outcome. Given this theoretical and empirical foundation, this thesis hypothesises that the KEDA-driven operator approach will exhibit measurably lower response latency under load spikes, fewer instances of cluster-level resource saturation, and a more stable Redis Cluster topology compared to the naive HPA baseline. The experimental evaluation presented in Chapter 6 is designed to test this hypothesis directly, using Grafana k6 to generate controlled, reproducible traffic patterns against both configurations and Prometheus-backed Grafana dashboards to measure the resulting autoscaling behaviour and service performance.

4 Technologies

This section provides a brief overview of each technology that constitutes the experimental infrastructure used in this thesis. Together these components form a reproducible, cloud-native testbed for evaluating the two autoscaling strategies under investigation.

4.1 K3s

K3s is a CNCF-certified, lightweight Kubernetes distribution developed by Rancher. It packages the full Kubernetes control plane into a single binary of less than 70 MB and eliminates several heavyweight dependencies of upstream Kubernetes, making it well-suited for resource-constrained environments such as edge devices, IoT deployments, and developer workstations (K3s Project, 2024). Despite its reduced footprint, K3s is fully conformant with the Kubernetes API and supports the complete set of Kubernetes primitives required by this thesis, including `StatefulSets`, `CustomResourceDefinitions`, and Helm chart deployments. K3s serves as the cluster management layer for all infrastructure described in this thesis.

4.2 FastAPI

FastAPI is a modern, high-performance Python web framework designed for building RESTful APIs. It is built on top of *Starlette* for its web-server layer and *Pydantic* for data validation, and it uses Python type hints to enable automatic request validation and interactive OpenAPI documentation generation (FastAPI, 2024b). FastAPI is built around the Asynchronous Server Gateway Interface (ASGI), enabling non-blocking I/O operations that are especially advantageous in workloads involving concurrent database or cache interactions (FastAPI, 2024a). In the context of this thesis, a FastAPI application serves as the HTTP entrypoint to the cluster, receiving incoming requests from the external load generator and routing read and write operations to the Redis Cluster backend.

4.3 Redis Cluster

Redis is an open-source, in-memory data structure store widely used as a cache, session store, and real-time data platform (Redis Ltd., 2024a). The Redis Cluster topology used in this thesis distributes data across multiple primary nodes using a hash-slot-based sharding mechanism. The full keyspace is divided into 16 384 hash slots, each assigned to a primary node, and each primary is optionally backed by one or more replica nodes for fault tolerance (Redis Ltd., 2024b). Redis Cluster provides high availability through automatic failover: if a primary becomes unreachable, one of its replicas is promoted. In this thesis, the Redis Cluster is deployed as a Kubernetes `StatefulSet` with a minimum of three primary nodes and forms the core stateful workload under performance evaluation.

4.4 Horizontal Pod Autoscaler (HPA)

The Horizontal Pod Autoscaler is a built-in Kubernetes controller that automatically adjusts the number of pod replicas in a workload based on observed resource metrics, most commonly CPU and memory utilisation (Kubernetes Documentation, 2024a). The HPA periodically queries the Kubernetes Metrics Server (or a custom metrics adapter) and computes a desired replica count proportional to the ratio of current to target utilisation. Its synchronisation loop runs every 15 seconds by default. While effective for many stateless use cases, the HPA is a fundamentally *reactive* mechanism: scaling decisions are always downstream of observed resource conditions, making it vulnerable to the lag issues described in Section 2.2. In this thesis, the HPA is deployed as the *naive* baseline scaling strategy, configured to scale the Redis `StatefulSet` in response to CPU and memory utilisation thresholds.

4.5 KEDA

Kubernetes Event-Driven Autoscaling (KEDA) is a CNCF-graduated open-source project that extends the Kubernetes HPA with the ability to scale workloads based on event sources and external metrics beyond simple CPU and memory (KEDA Project, 2024a). KEDA introduces the `ScaledObject` custom resource, which declares a scaling target (such as a `Deployment` or `StatefulSet`), a polling interval, cooldown periods, and one or more *scalers* that define the metric source (KEDA Project, 2024c). When the configured metric from the scaler exceeds a threshold, KEDA adjusts the replica count of the target workload by updating the underlying HPA on the user's behalf. In this thesis, KEDA is configured with a Prometheus scaler that reads application-level metrics — such as request queue depth and cache hit ratio — directly from the Prometheus monitoring stack, enabling proactive, context-aware scaling decisions before resource exhaustion occurs (KEDA Project, 2024b).

4.6 Prometheus and Grafana

The observability stack in this thesis is deployed using the `kube-prometheus-stack` Helm chart, which bundles Prometheus, Grafana, and Alertmanager into a single, opinionated release (Prometheus Community, 2024). *Prometheus* is an open-source monitoring system and time-series database that collects metrics by periodically scraping HTTP endpoints exposed by instrumented targets (Prometheus Authors, 2024). In the Kubernetes context, it discovers scrape targets automatically via Kubernetes service discovery, collecting cluster-level metrics from `kube-state-metrics` and `node-exporter`, as well as application-level metrics from the FastAPI and Redis exporters. *Grafana* is an open-source analytics and visualisation platform that queries Prometheus via its native data source integration and renders the collected time-series data as configurable dashboards (Grafana Labs, 2024b). Grafana dashboards are used throughout the experimental evaluation in this thesis to visualise autoscaling behaviour, latency distributions, and resource utilisation over time. Crucially, the Prometheus instance also acts as the external metric backend for the KEDA Prometheus scaler, creating a unified

data path from application telemetry to autoscaling decisions.

4.7 Helm

Helm is the de-facto package manager for Kubernetes. It introduces the concept of a *chart* — a collection of templated Kubernetes manifests that describe a deployable application — and a *release*, which is a running instance of a chart in a cluster (Helm Project, 2024). Helm charts enable repeatable, version-controlled deployments with configurable values overrides and built-in rollback support, significantly reducing the operational complexity of deploying multi-resource applications such as the `kube-prometheus-stack`. In this thesis, Helm is used to deploy the observability stack, and the deployment configurations are parameterised through values files that are committed alongside the thesis infrastructure code.

4.8 Grafana k6

Grafana k6 is an open-source, developer-focused performance and load testing tool (Grafana Labs, 2024c). Tests in k6 are written as JavaScript modules, and k6 executes them using a Go runtime which maps each *virtual user* (VU) to a lightweight goroutine, enabling high concurrency with low resource overhead (Grafana Labs, 2024d). k6 supports configurable test scenarios including ramp-up and ramp-down of virtual users over time, making it possible to simulate realistic traffic patterns such as gradual load increases, sustained peaks, and sudden spikes (Grafana Labs, 2024a). In the context of this thesis, k6 runs *outside* the Kubernetes cluster as an independent load generator. It sends configurable volumes of HTTP traffic through the FastAPI endpoint and into the Redis Cluster, producing the synthetic workload against which both autoscaling strategies — HPA and KEDA — are evaluated. The ability to precisely control the number of virtual users and the ramp profile makes k6 well-suited for generating reproducible, parameterised load scenarios that can isolate the performance characteristics of each scaling approach.

5 Experiment Architecture

6 Results and Analysis

7 Conclusion

Anhang

Hier steht der Anhang.

Glossar

Hier steht das Glossar.

Literaturverzeichnis

- Amazon Web Services. (2022). *Container Build Lens — AWS Well-Architected Framework: Design Principles* [Accessed: 2026-01-24]. <https://docs.aws.amazon.com/pdfs/wellarchitected/latest/container-build-lens/container-build-lens.pdf#design-principles>
- Cloud Native Computing Foundation. (2023). *CNCF Announces Graduation of KEDA* [Accessed: 2026-01-24]. <https://www.cncf.io/announcements/2023/08/22/cloud-native-computing-foundation-announces-graduation-of-kubernetes-autoscaler-keda/>
- Cloud Native Computing Foundation. (2024). *Managing Large-Scale Redis Clusters on Kubernetes with an Operator* [Accessed: 2026-01-24]. <https://www.cncf.io/blog/2024/12/17/managing-large-scale-redis-clusters-on-kubernetes-with-an-operator-kuaishous-approach/>
- CNCF TAG App Delivery. (2023). *CNCF Operator White Paper* [Accessed: 2026-01-24]. <https://tag-app-delivery.cncf.io/whitepapers/operator/>
- Docker Inc. (2016). *Containers are not VMs* [Accessed: 2026-01-24]. <https://www.docker.com/blog/containers-are-not-vm/>
- FastAPI. (2024a). *Concurrency and async / await — FastAPI* [Accessed: 2026-01-24]. <https://fastapi.tiangolo.com/async/>
- FastAPI. (2024b). *FastAPI — Modern, Fast, Web Framework for Building APIs with Python* [Accessed: 2026-01-24]. <https://fastapi.tiangolo.com/>
- Google Cloud. (2023). *Zero-downtime Scaling in Memorystore for Redis Cluster* [Accessed: 2026-01-24]. <https://cloud.google.com/blog/products/databases/zero-downtime-scaling-in-memorystore-for-redis-cluster>
- Grafana Labs. (2024a). *API Load Testing — Grafana k6 Documentation* [Accessed: 2026-01-24]. <https://grafana.com/docs/k6/latest/testing-guides/api-load-testing/>
- Grafana Labs. (2024b). *Grafana — The Open and Composable Observability Platform* [Accessed: 2026-01-24]. <https://github.com/grafana/grafana>
- Grafana Labs. (2024c). *Grafana k6 Documentation* [Accessed: 2026-01-24]. <https://grafana.com/docs/k6/latest/>
- Grafana Labs. (2024d). *k6 — A Modern Load Testing Tool* [Accessed: 2026-01-24]. <https://github.com/grafana/k6>
- Helm Project. (2024). *Using Helm* [Accessed: 2026-01-24]. https://helm.sh/docs/intro/using_helm/
- IBM. (2024). *The History of Kubernetes* [Accessed: 2026-01-24]. <https://www.ibm.com/think/topics/kubernetes-history>
- K3s Project. (2024). *K3s — Lightweight Kubernetes* [Accessed: 2026-01-24]. <https://docs.k3s.io/>
- KEDA Project. (2024a). *KEDA — Kubernetes Event-driven Autoscaling* [Accessed: 2026-01-24]. <https://keda.sh/>
- KEDA Project. (2024b). *KEDA — Prometheus Scaler* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/scalers/prometheus/>
- KEDA Project. (2024c). *KEDA — ScaledObject Specification* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/reference/scaledobject-spec/>

7. Conclusion

- KEDA Project. (2024d). *KEDA — Scaling Deployments, StatefulSets & Custom Resources* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/concepts/scaling-deployments/>
- Kubernetes Documentation. (2024a). *Autoscaling Workloads* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/autoscaling/>
- Kubernetes Documentation. (2024b). *Disruptions — Pod Disruption Budgets* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/pods/disruptions/>
- Kubernetes Documentation. (2024c). *Kubernetes Overview* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/overview/>
- Kubernetes Documentation. (2024d). *Operator Pattern* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- Kubernetes Documentation. (2024e). *StatefulSets* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
- Medium / emreblblvv. (2024). *Auto-scaling with KEDA Using Custom RED Metrics from Prometheus* [Accessed: 2026-01-24]. <https://medium.com/@emreblblvv/auto-scaling-with-keda-using-custom-red-metrics-from-prometheus-76d50785e442>
- Mell, P. & Grance, T. (2011). *The NIST Definition of Cloud Computing* (Techn. Ber. Special Publication 800-145) [Accessed: 2026-01-24]. National Institute of Standards and Technology. Accessed: 2026-01-24. <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>
- Portworx. (2024). *Understanding StatefulSets in Kubernetes* [Accessed: 2026-01-24]. <https://portworx.com/knowledge-hub/understanding-statefulsets-in-kubernetes/>
- Prometheus Authors. (2024). *Prometheus Overview* [Accessed: 2026-01-24]. <https://prometheus.io/docs/introduction/overview/>
- Prometheus Community. (2024). *kube-prometheus-stack Helm Chart* [Accessed: 2026-01-24]. <https://artifacthub.io/packages/helm/prometheus-community/kube-prometheus-stack>
- Red Hat. (2025). *Stateful vs. Stateless Applications* [Accessed: 2026-01-24]. <https://www.redhat.com/en/topics/cloud-native-apps/stateful-vs-stateless>
- Redis Ltd. (2024a). *Redis — Real-time Data for Agents & Apps* [Accessed: 2026-01-24]. <https://redis.io/>
- Redis Ltd. (2024b). *Redis Cluster Specification* [Accessed: 2026-01-24]. https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
- Thoras. (2024). *Predictive Horizontal Pod Autoscaling* [Accessed: 2026-01-24]. <https://docs.thoras.ai/guides/hpa>
- Vazquez, A. (2026). *Kubernetes HPA Best Practices: When CPU Works, Why Memory Doesn't* [Accessed: 2026-01-24]. <https://alexandre-vazquez.com/kubernetes-hpa-best-practices/>
- Wanigasooriya, C. & Ekanayake, I. (2026). *NimbusGuard: A Novel Framework for Proactive Kubernetes Autoscaling Using Deep Q-Networks* [Accessed: 2026-01-24]. *arXiv preprint arXiv:2604.11017*. <https://arxiv.org/html/2604.11017v1>

Stichwortverzeichnis

Hier steht das Stichwortverzeichnis.

Anlagen

Hier sind die Anlagen.